

Types of JOIN in SQL

Lab 4

- Inner Join

```
CREATE DATABASE IF NOT EXISTS store_db;  
USE store_db;
```

```
DROP TABLE IF EXISTS Orders;  
DROP TABLE IF EXISTS Users;
```

```
CREATE TABLE Users (  
    user_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    email VARCHAR(100)  
);
```

```
CREATE TABLE Orders (  
    order_id INT PRIMARY KEY,  
    user_id INT,  
    product_name VARCHAR(100),  
    amount DECIMAL(10, 2),  
    FOREIGN KEY (user_id) REFERENCES Users(user_id)  
);
```

```
INSERT INTO Users (user_id, name, email) VALUES  
(1, 'Sanjana', 'sanjana@gmail.com'),  
(2, 'Sahasra', 'sahasra@gmail.com'),  
(3, 'Ninada', 'ninada@gmail.com');
```

```
INSERT INTO Orders (order_id, user_id, product_name, amount) VALUES  
(101, 1, 'Smartphone', 699.99),  
(102, 1, 'Headphones', 150.00),  
(103, 2, 'Smartwatch', 249.50);
```

```
SELECT  
    Users.user_id,
```

```

Users.name,
Users.email,
Orders.order_id,
Orders.product_name,
Orders.amount
FROM Users
INNER JOIN Orders ON Users.user_id = Orders.user_id;

```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 						
	user_id	name	email	order_id	product_name	amount
▶	1	Sanjana	sanjana@gmail.com	101	Smartphone	699.99
	1	Sanjana	sanjana@gmail.com	102	Headphones	150.00
	2	Sahasra	sahasra@gmail.com	103	Smartwatch	249.50

- Left Join

```

CREATE DATABASE IF NOT EXISTS store_db;
USE store_db;

```

```

DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Users;

```

```

CREATE TABLE Users (
    user_id INT PRIMARY KEY,
    name VARCHAR(50),
    email VARCHAR(100)
);

```

```

CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    user_id INT,
    product_name VARCHAR(100),
    amount DECIMAL(10, 2),
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);

```

```

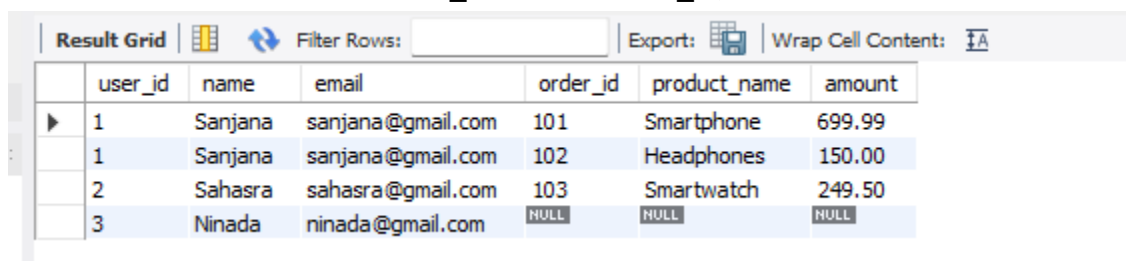
INSERT INTO Users (user_id, name, email) VALUES

```

```
(1, 'Sanjana', 'sanjana@gmail.com'),  
(2, 'Sahasra', 'sahasra@gmail.com'),  
(3, 'Ninada', 'ninada@gmail.com');
```

```
INSERT INTO Orders (order_id, user_id, product_name, amount) VALUES  
(101, 1, 'Smartphone', 699.99),  
(102, 1, 'Headphones', 150.00),  
(103, 2, 'Smartwatch', 249.50);
```

```
SELECT  
    Users.user_id,  
    Users.name,  
    Users.email,  
    Orders.order_id,  
    Orders.product_name,  
    Orders.amount  
FROM Users  
LEFT JOIN Orders ON Users.user_id = Orders.user_id;
```



The screenshot shows a database query result grid with the following data:

	user_id	name	email	order_id	product_name	amount
▶	1	Sanjana	sanjana@gmail.com	101	Smartphone	699.99
	1	Sanjana	sanjana@gmail.com	102	Headphones	150.00
	2	Sahasra	sahasra@gmail.com	103	Smartwatch	249.50
	3	Ninada	ninada@gmail.com	NULL	NULL	NULL

- Outer Left Join

```
CREATE DATABASE IF NOT EXISTS store_db;  
USE store_db;
```

```
DROP TABLE IF EXISTS Orders;  
DROP TABLE IF EXISTS Users;
```

```
CREATE TABLE Users (  
    user_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    email VARCHAR(100)  
);
```

```
CREATE TABLE Orders (  
    order_id INT PRIMARY KEY,  
    user_id INT,  
    product_name VARCHAR(100),  
    amount DECIMAL(10, 2)  
);
```

```
INSERT INTO Users (user_id, name, email) VALUES  
(1, 'Sanjana', 'sanjana@gmail.com'),  
(2, 'Sahasra', 'sahasra@gmail.com'),  
(3, 'Ninada', 'ninada@gmail.com');
```

```
INSERT INTO Orders (order_id, user_id, product_name, amount) VALUES  
(101, 1, 'Smartphone', 699.99),  
(102, 1, 'Headphones', 150.00),  
(103, 2, 'Smartwatch', 249.50),  
(104, 4, 'Tablet', 300.00);
```

```
SELECT  
    Users.user_id,  
    Users.name,  
    Users.email,  
    Orders.order_id,  
    Orders.product_name,  
    Orders.amount  
FROM  
    Users  
    LEFT JOIN  
    Orders ON Users.user_id = Orders.user_id  
UNION SELECT  
    Users.user_id,  
    Users.name,  
    Users.email,  
    Orders.order_id,  
    Orders.product_name,  
    Orders.amount  
FROM  
    Users  
    RIGHT JOIN  
    Orders ON Users.user_id = Orders.user_id;
```

Result Grid						
		Filter Rows:		Export:	Wrap Cell Content:	
	user_id	name	email	order_id	product_name	amount
▶	1	Sanjana	sanjana@gmail.com	102	Headphones	150.00
	1	Sanjana	sanjana@gmail.com	101	Smartphone	699.99
	2	Sahasra	sahasra@gmail.com	103	Smartwatch	249.50
	3	Ninada	ninada@gmail.com	NULL	NULL	NULL
	NULL	NULL	NULL	104	Tablet	300.00

Result 1 x

- Cross Join

```
CREATE DATABASE IF NOT EXISTS store_db;
USE store_db;
```

```
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Users;
```

```
CREATE TABLE Users (
  user_id INT PRIMARY KEY,
  name VARCHAR(50),
  email VARCHAR(100)
);
```

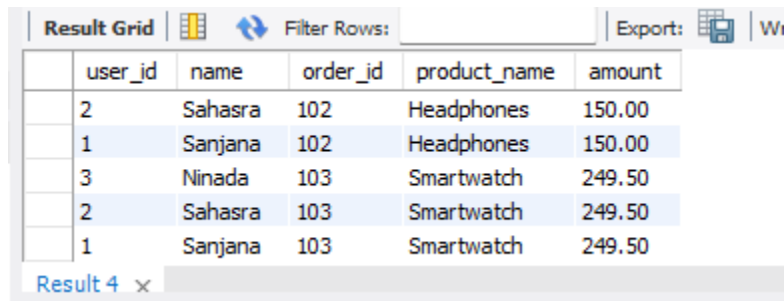
```
CREATE TABLE Orders (
  order_id INT PRIMARY KEY,
  user_id INT,
  product_name VARCHAR(100),
  amount DECIMAL(10, 2)
);
```

```
INSERT INTO Users (user_id, name, email) VALUES
(1, 'Sanjana', 'sanjana@gmail.com'),
(2, 'Sahasra', 'sahasra@gmail.com'),
(3, 'Ninada', 'ninada@gmail.com');
```

```
INSERT INTO Orders (order_id, user_id, product_name, amount) VALUES
```

```
(101, 1, 'Smartphone', 699.99),  
(102, 1, 'Headphones', 150.00),  
(103, 2, 'Smartwatch', 249.50);
```

```
SELECT  
    Users.user_id,  
    Users.name,  
    Orders.order_id,  
    Orders.product_name,  
    Orders.amount  
FROM Users  
CROSS JOIN Orders;
```



The screenshot shows a database interface with a 'Result Grid' tab. It displays the results of a SQL query. The grid has columns for user_id, name, order_id, product_name, and amount. There are five rows of data, representing the cross join of the Users and Orders tables. The first row shows user_id 2, name Sahasra, order_id 102, product_name Headphones, and amount 150.00. The second row shows user_id 1, name Sanjana, order_id 102, product_name Headphones, and amount 150.00. The third row shows user_id 3, name Ninada, order_id 103, product_name Smartwatch, and amount 249.50. The fourth row shows user_id 2, name Sahasra, order_id 103, product_name Smartwatch, and amount 249.50. The fifth row shows user_id 1, name Sanjana, order_id 103, product_name Smartwatch, and amount 249.50. The interface also includes a 'Filter Rows' section and an 'Export' button.

	user_id	name	order_id	product_name	amount
	2	Sahasra	102	Headphones	150.00
	1	Sanjana	102	Headphones	150.00
	3	Ninada	103	Smartwatch	249.50
	2	Sahasra	103	Smartwatch	249.50
	1	Sanjana	103	Smartwatch	249.50

Result 4 x

- **Subquery (Find teachers whose salary is greater than the average salary)**

```
CREATE DATABASE IF NOT EXISTS school_db;  
USE school_db;
```

```
DROP TABLE IF EXISTS Teachers;
```

```
CREATE TABLE Teachers (  
    teacher_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    email VARCHAR(100),  
    subject VARCHAR(50),  
    salary DECIMAL(10, 2)  
);
```

```
INSERT INTO Teachers (teacher_id, name, email, subject, salary) VALUES  
(1, 'Sanjana', 'sanjana@school.edu', 'Mathematics', 65000.00),  
(2, 'Sahasra', 'sahasra@school.edu', 'Science', 72000.00),
```

```
(3, 'Ninada', 'ninada@school.edu', 'English', 58000.00);
```

-- 5. Subquery: Find teachers whose salary is greater than the average salary

```
SELECT
    teacher_id,
    name,
    subject,
    salary
FROM Teachers
WHERE salary > (
    SELECT AVG(salary)
    FROM Teachers
);
```

	teacher_id	name	subject	salary
▶	2	Sahasra	Science	72000.00
•	NULL	NULL	NULL	NULL